

Stage 2 Report:

Predicting Helsinki City Bike Trip Duration from Route, Time and Weather

CS-C3240 Machine Learning

1. Introduction

Shared city bikes are popular in Helsinki and Espoo, but riders cannot easily tell in advance how long a trip will take. A good estimate would help people decide whether to cycle, for example in bad weather or at a busy time of day. This project predicts how long a city bike trip will take before it starts. Section 2 defines the problem and the data, Section 3 describes our methods, Section 4 presents the results, and Section 5 summarises what we learned.

2. Problem Formulation

This is a supervised regression problem. Each data point is one completed bike trip, and the label is the trip's duration in minutes. Each trip is described by seven features (Table 1): its route, when it starts, and the weather that day.

Table 1. Features describing each trip.

Feature	Type	Description
Route length	Continuous	Typical distance (m) between the start and end station, from past trips
Round trip	Binary	Whether the bike is returned to the station it was taken from
Weekday	Categorical	Day the trip started (Mon-Sun)
Hour of day	Categorical	Hour the trip started (0-23)
Month	Categorical	Month of the season (Apr-Oct)
Temperature	Continuous	Daily mean air temperature (°C)
Precipitation	Continuous	Daily total precipitation (mm)

The problem is the same as in Stage 1, but we added the route. In Stage 1 we left out distance entirely, so that the model could not see the answer. On reflection, that mixed up two different things. The distance a trip actually covered is only known after the ride, so we still leave it out. But riders know where they are going before they start, so the typical length of their route is fair to use.

The trip data comes from HSL's open dataset of city bike trips [1]. The weather data comes from the Finnish Meteorological Institute's open data for the Kaisaniemi station in Helsinki [2], and is matched to each trip by date.

3. Methods

3.1 Data and preprocessing

We use all trips from the April-October seasons of 2022-2025, about 10 million in total. We removed incomplete rows, "false starts" shorter than 10 seconds or 10 metres (a bike taken out and put straight back), and bikes moved to HSL's workshop or test stations. This leaves **9,837,352 trips**. We kept very long trips, most of which are bikes that were never properly returned, because there is no natural cut-off for removing them. Section 3.4 explains how we handle them instead.

3.2 Feature selection

We started from three assumptions: longer trips take longer, the time of day affects speed (through traffic or the kind of trip), and bad weather slows riders down. Since duration is distance divided by speed, we plotted the typical speed and the typical distance against each factor. Distance varied far more than speed, so trip length is the main thing to capture. Speed changed somewhat with the time of day: riders were fastest in the morning, when many of them are commuting, and slower in the afternoon. This suggests that the purpose of the trip matters more than traffic. Rain did not slow riders down, contrary to our Stage 1 assumption; on rainy days, people simply made shorter trips. We kept the Stage 1 time and weather features so that we could test their effect, and added two route features.

Route length is the median distance covered by other trips between the same two stations. To avoid leaking information, it is only computed from other seasons: 2022 training trips use 2023 trips and vice versa, validation trips use 2022-2023, and test trips use 2022-2024. Routes with fewer than three past trips use the median for their starting station instead. The **round-trip** flag marks trips that end where they started, which are mostly leisure rides.

3.3 Models

We compare two hypothesis spaces covered in the course [3]:

- **Linear regression.** We predict the logarithm of duration from the logarithm of route length and the other features. This matches the idea that duration is distance divided by speed, where time and weather make riders a few percent faster or slower.
- **Gradient-boosted regression trees** [5]. Trees can capture interactions that a linear model cannot, for example rain mattering more on long trips. We include them to check whether such interactions exist.

To see how much the models actually learn, we compare them with two simple baselines: always predicting the median duration, and a **physics baseline** that divides route length by one typical speed (11.5 km/h, estimated from the training data).

3.4 Loss functions

A small number of trips last hours or even days, mostly bikes that were never returned. With the usual squared error loss, these few trips would dominate training: trips longer than five hours are only 0.3% of the data, but they make up almost all of the squared error. We handle this in two ways. First, we predict the logarithm of the duration, which shrinks these extreme values. Second, we train the linear model with the **Huber loss** [4] ($\epsilon = 1.35$). It treats small errors like the squared error, but large errors only linearly, so a few extreme trips cannot pull the model too far. To check whether this matters, we also train the same linear model with the **squared error**. For the trees we use the **absolute error**, which is also robust to outliers; the library we used does not offer the Huber loss for trees.

This gives three methods: linear regression with squared error, linear regression with Huber loss, and trees with absolute error. We evaluate them in minutes, using the mean absolute error (MAE), the median absolute error (MedAE, the error on a typical trip) and the share of trips predicted within two minutes.

3.5 Training, validation and test sets

We split the data by season: 2022-2023 for training (4,876,755 trips), 2024 for validation (2,508,741 trips) and 2025 for testing (2,451,856 trips). This mimics real use, where the model predicts trips in a future season. A random split would put trips from the same days into both training and validation, which would make the results look better than they are. To keep training fast, every model is fitted on the same random sample of 400,000 training trips. We decided in advance to choose the method with the lowest validation MAE, retrain it on 2022-2024, and evaluate it once on the test set. Because the differences between methods are small, we also compute 95% confidence intervals for them by resampling whole days [6].

4. Results

4.1 Comparing the methods

Table 2. Errors in minutes, and share of trips predicted within 2 minutes. The chosen method is in bold.

Method	Training		Validation (2024)			Test (2025)		
	MAE	MedAE	MAE	MedAE	≤ 2 min	MAE	MedAE	≤ 2 min
Constant (median)	10.40	4.88	13.98	5.30	19.0%	14.89	5.43	18.6%
Physics baseline	7.92	1.76	11.22	2.06	49.1%	11.95	2.14	48.0%
Linear, squared error	7.74	1.98	11.00	2.21	46.1%	11.70	2.35	43.3%
Linear, Huber loss	7.62	1.62	10.91	1.86	52.5%	11.55	1.91	51.5%
Trees, absolute error	7.57	1.58	10.96	1.85	52.5%	11.60	1.89	51.8%

Table 2 shows four things:

- **Route length does most of the work.** The physics baseline only divides route length by a typical speed, yet it already cuts the typical validation error from 5.3 to 2.1 minutes.
- **Machine learning adds a small but real improvement.** The best methods bring the typical error down to about 1.85 minutes and predict 3.3 percentage points more trips within two minutes than the physics baseline (95% CI [3.1, 3.5]). Part of the gain comes from details the simple rule misses: longer routes are ridden faster, so doubling a route makes a trip about 80% longer rather than twice as long.
- **The Huber loss matters.** With squared error, the few extreme trips still pull predictions up by about half a minute, so this method does worse than the physics baseline on typical trips. With the Huber loss, 6.4 percentage points more trips are predicted within two minutes (CI [6.1, 6.6]). Removing the longest 0.1% of training trips also moves the squared-error model's predictions about ten times more than the Huber model's.
- **Trees do not help.** The trees and the Huber model are practically tied. The linear model's MAE is 0.05 minutes (about three seconds) lower, and both predict the same share of trips within two minutes.

Training errors are lower than validation errors for every method, including the constant baseline, which learns almost nothing. The gap therefore comes from 2024 being a harder season, not from overfitting. We choose **linear regression with the Huber loss**: it has the lowest validation MAE, as our rule requires, and it is simpler and easier to interpret than the trees.

4.2 Which features matter

To test our three assumptions, we refitted the chosen model with different sets of features (Table 3).

Table 3. Validation results of the chosen model with different features. "Route" means route length and the round-trip flag.

Features used by the chosen model	Validation MedAE (min)	Within 2 min
None (constant prediction)	5.300	19.0%
Time + weather (the Stage 1 features)	5.250	19.1%
Route	1.865	52.3%
Route + time	1.850	52.6%
Route + weather	1.875	52.2%
Route + time + weather (chosen model)	1.857	52.5%

Route length explains almost everything. Without it, the Stage 1 features barely beat a constant prediction; with route length alone, the model is almost as good as the full model. Time adds a little: on the same route, a trip takes about 10% less time at 5-6 am and about 4% more in the early afternoon than on average, and weekend trips take about 3% longer. Weather adds nothing, and adding it to route length even makes the typical error slightly worse. In the fitted model, 10 mm of rain makes a trip only about 1% shorter.

4.3 Test error

The 2025 test season was not used for training or for choosing the method. On it, the chosen method has a **test MAE of 11.55 minutes and a test MedAE of 1.91 minutes**, and predicts 51.5% of trips within two minutes (Table 2). This is close to its validation results, so the model also works on a new season, and the methods rank the same way as on the validation data. The predictions are not biased: on a typical trip, the ratio of actual to predicted duration is 1.006.

5. Conclusions

We predicted city bike trip durations from the route, the time and the weather. The chosen method, linear regression with the Huber loss, has a typical error of 1.9 minutes on an unseen season, against 5.4 minutes for a constant guess. The main lesson is that route length explains most of the prediction: a simple distance-divided-by-speed rule already does well, and machine learning adds a small but real improvement on top. Training with the Huber loss matters because of the extreme trips, while more complex trees and the weather add little.

For a typical trip, the problem is solved well. The average error (about 11.5 minutes) stays much larger than the typical error because it is dominated by bikes that were never returned and by riders who stop along the way, which our features cannot predict. The results could be improved with hourly instead of daily weather, station coordinates to estimate the length of routes without history, training on all trips instead of a sample, and a separate model that detects bikes that were not returned.

Use of AI

We used generative AI (Claude, by Anthropic) throughout Stage 2. It helped us plan and carry out the analyses, write and debug the notebook code, think through the problem formulation (including the difference between a trip's recorded distance and its route), and draft and edit this report. We made the main design decisions ourselves: adding route length, comparing three methods, and computing route length from past trips. We checked the code, results and text before submitting.

6. References

- [1] HSL (Helsinki Region Transport). Helsinki and Espoo city bike trip data (CC BY 4.0). https://hri.fi/data/en_GB/dataset/helsingin-ja-espoon-kaupunkipyorilla-ajatut-matkat
- [2] Finnish Meteorological Institute. Open data, weather observations (CC BY 4.0). <https://en.ilmatieteenlaitos.fi/open-data>
- [3] Jung, A. Machine Learning: The Basics. <https://mlbook.cs.aalto.fi>
- [4] Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1), 73-101.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [6] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

7. Appendix: Code

The full notebook (ML_bike.ipynb), which prepares the data and trains and evaluates all methods, is attached on the following pages.

ML-bike: predicting city-bike trip duration from route, time and weather

Goal: before a trip starts, predict how long it will take — from the route the rider has chosen, when they set off, and what the weather is like.

ML problem formulation (Stage 2)

- **Data point:** one bike trip.
- **Label:** trip duration in minutes. Models are fitted on $\log(\text{duration})$, so every feature acts as a *percentage* change in duration rather than a fixed number of minutes.
- **Features:**
 - **route length** — the typical distance ridden between the chosen start and end station, learned from earlier seasons
 - **round trip** — whether the bike goes back to the station it came from
 - **time** — hour of day, weekday, month
 - **weather** — daily mean temperature and daily precipitation
- **Hypothesis spaces:** linear regression on one-hot encoded calendar features; gradient-boosted regression trees.
- **Loss:** squared error and Huber loss for the linear model; absolute error for the trees.
- **Validation:** time-based split. Train on 2022–2023, validate on 2024, test once on 2025.

What changed since Stage 1

Stage 1 excluded distance so that the model could not "cheat". On review, that argument mixed up two different quantities:

- the **distance recorded for the trip itself** is only known after the ride ends, so it stays excluded;
- the **route** — which two stations the rider is travelling between — is known before departure, and our problem statement already assumes the rider knows their destination.

Without the route, time and weather alone beat a constant prediction by under 1% (step 11 shows this). Stage 2 therefore adds route length, learned only from earlier seasons, so it describes the route rather than the trip being predicted.

Stage 2 also narrows the comparison to three models, chosen so that each pair differs in exactly one thing (step 8).

Data

- `data/YYYY-MM.csv` — trips from the Helsinki & Espoo city-bike system, seasons April–October 2022–2025.
- `data/weather_daily.csv` — daily mean temperature and precipitation from the Finnish Meteorological Institute (FMI) open data, station Helsinki Kaisaniemi. Step 3

shows how it was downloaded.

Steps:

1. Setup
2. Load the trips
3. Load the weather
4. Merge and clean
5. Explore: the outlier problem
6. Explore: duration = distance ÷ speed
7. Features and train/validation/test split
8. Three models and two reference points
9. Are the differences real? A day-block bootstrap
10. Does the loss still matter? An outlier-sensitivity experiment
11. Testing the three-factor assumption: feature ablation
12. What the model learned
13. Final evaluation on the 2025 test season
14. Conclusions

1. Setup

In [1]:

```
import ssl
import urllib.parse
import urllib.request
import warnings
import xml.etree.ElementTree as ET
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import HuberRegressor, LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

DATA_DIR = Path("data")
WEATHER_CSV = DATA_DIR / "weather_daily.csv"
RANDOM_STATE = 42
TRAIN_SAMPLE = 400_000

pd.set_option("display.float_format", "{:,.3f}".format)
plt.rcParams["figure.figsize"] = (11, 4)
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.alpha"] = 0.3
```

2. Load the trips

The monthly files all share the same columns. We need:

- the **departure time**, for the calendar features and for joining weather by date;
- the **departure and return station**, which together define the route;
- the **duration**, our label;
- the **recorded distance**.

The recorded distance is never a feature. It is used to filter false starts and — from earlier seasons only — to learn how long each route typically is (step 7).

In [2]:

```
COLUMNS = {
    "Departure": "departure",
    "Departure station id": "from_id",
    "Return station id": "to_id",
    "Covered distance (m)": "distance_m",
    "Duration (sec.)": "duration_s",
}

files = sorted(DATA_DIR.glob("[0-9]*.csv"))
print(f"Found {len(files)} monthly trip files")

frames = []
for f in files:
    month_df = pd.read_csv(
        f,
        usecols=list(COLUMNS),
        dtype={
            "Departure station id": "str",
            "Return station id": "str",
            "Covered distance (m)": "float32",
            "Duration (sec.)": "float32",
        },
    ).rename(columns=COLUMNS)
    frames.append(month_df)

trips = pd.concat(frames, ignore_index=True)
del frames

# A few rows hold only a date with no time of day; they become NaT and are dropped
in step 4.
trips["departure"] = pd.to_datetime(trips["departure"], format="%Y-%m-%dT%H:%M:%S", errors="coerce")

print(f"Loaded {len(trips):,} trips")
trips.head()
```

Found 28 monthly trip files

Loaded 10,121,026 trips

Out[2]:

	departure	from_id	to_id	distance_m	duration_s
0	2022-04-30 23:59:53	001	004	1,204.000	128,982.000
1	2022-04-30 23:59:39	710	707	559.000	284.000
2	2022-04-30 23:59:19	403	131	2,199.000	1,078.000
3	2022-04-30 23:59:03	039	086	1,933.000	590.000
4	2022-04-30 23:58:55	002	012	1,208.000	477.000

3. Load the weather

`data/weather_daily.csv` holds daily observations from FMI's open data WFS API (station Helsinki Kaisaniemi, `fmid=100971`):

- `tday` → `temp_c`, daily mean temperature in °C
- `rrday` → `precip_mm`, daily precipitation in mm

FMI reports **-1 mm** when no measurable precipitation fell, so we clip those to 0.

The cell below downloads the file if it is missing, so the notebook can be rerun from scratch. Daily weather is a compromise: hourly observations would match trips better, but a daily join keeps the data pipeline simple.

In [3]:

```
FMID = 100971 # Helsinki Kaisaniemi

def download_weather(path: Path, years=range(2022, 2026)) -> None:
    import certifi # provides CA certificates; some Python installs cannot verify
    TLS without it

    context = ssl.create_default_context(cafile=certifi.where())
    ns = "{http://xml.fmi.fi/schema/wfs/2.0}"
    frames = []

    for year in years:
        query = urllib.parse.urlencode({
            "service": "WFS",
            "version": "2.0.0",
            "request": "getFeature",
            "storedquery_id": "fmi::observations::weather::daily::simple",
            "fmid": FMID,
            "starttime": f"{year}-04-01T00:00:00Z",
            "endtime": f"{year}-10-31T23:59:59Z",
            "parameters": "tday,rrday",
        })
        with urllib.request.urlopen(f"https://opendata.fmi.fi/wfs?{query}",
            timeout=120, context=context) as r:
            root = ET.fromstring(r.read())

        rows = [
            {
                "time": el.findtext(f"{ns}Time"),
```



```

        "name": el.findtext(f"{ns}ParameterName"),
        "value": pd.to_numeric(el.findtext(f"{ns}ParameterValue")),
errors="coerce"),
    }
    for el in root.iter(f"{ns}BSWfsElement")
    ]
    wide = pd.DataFrame(rows).pivot_table(index="time", columns="name",
values="value", aggfunc="first")
    wide.index = pd.to_datetime(wide.index).tz_convert(None).normalize()
    frames.append(wide)
    print(f"  downloaded {year}: {len(wide)} days")

    out = pd.concat(frames).sort_index().rename(columns={"tday": "temp_c",
"rrday": "precip_mm"})
    out.index.name = "date"
    out[["temp_c", "precip_mm"]].to_csv(path, float_format="%.1f")

if not WEATHER_CSV.exists():
    print("Downloading weather from FMI open data...")
    download_weather(WEATHER_CSV)

weather = pd.read_csv(WEATHER_CSV, parse_dates=
["date"]).set_index("date").sort_index()
weather["precip_mm"] = weather["precip_mm"].clip(lower=0) # -1 means "no
measurable rain"

print(f"{len(weather)} days, {weather.index.min().date()} ->
{weather.index.max().date()}")
print(f"Rainy days (>= 0.1 mm): {(weather['precip_mm'] >= 0.1).mean():.1%}")
weather.describe()

```

856 days, 2022-04-01 -> 2025-10-31

Rainy days (>= 0.1 mm): 39.6%

Out[3]:

	temp_c	precip_mm
count	856.000	856.000
mean	13.035	1.892
std	5.847	4.867
min	-2.000	0.000
25%	8.800	0.000
50%	13.750	0.000
75%	17.625	1.200
max	24.900	43.100

4. Merge and clean

We join weather onto trips by calendar date, then apply a **minimal** cleaning rule:

- drop missing values and **false starts** — under 10 seconds or under 10 metres, a bike undocked and immediately re-docked;
- drop moves to or from HSL's **workshop and test stations** (997 , 999). These are maintenance and system tests, not rides.

We deliberately **keep the long trips**, forgotten rentals included. Deleting them needs an arbitrary cut-off; handling them through the label transform and the loss is the choice this project defends.

Finally, each station gets an integer code, and each (start, end) pair becomes a **route**.

In [4]:

```
NON_RIDE_STATIONS = {"997": "Workshop Helsinki", "999": "Relay Box test station / Station Test"}

n_raw = len(trips)

trips = trips.dropna(subset=["departure", "duration_s", "distance_m", "from_id", "to_id"])
trips["from_id"] = trips["from_id"].str.strip()
trips["to_id"] = trips["to_id"].str.strip()

trips["date"] = trips["departure"].dt.normalize()
trips = trips.merge(weather, left_on="date", right_index=True, how="left")

no_weather = trips["temp_c"].isna()
print(f"Trips with no matching weather day:      {no_weather.sum():,}")
trips = trips[~no_weather]

false_start = (trips["duration_s"] < 10) | (trips["distance_m"] < 10)
non_ride = trips["from_id"].isin(NON_RIDE_STATIONS) |
trips["to_id"].isin(NON_RIDE_STATIONS)
print(f"False starts removed:                  {false_start.sum():,}")
print(f"Workshop / test-station moves removed: {(non_ride &
~false_start).sum():,}")
trips = trips[~false_start & ~non_ride].reset_index(drop=True)

# float64: the log transform in step 8 checks that exp(log(y)) == y, which fails
at float32 precision.
trips["duration_min"] = trips["duration_s"].astype("float64") / 60
trips["distance_km"] = trips["distance_m"].astype("float64") / 1000

# Integer station codes; a route is an ordered (start, end) pair.
stations = pd.Index(pd.unique(np.concatenate([trips["from_id"].to_numpy(),
trips["to_id"].to_numpy()])))
trips["from_code"] = stations.get_indexer(trips["from_id"]).astype("int32")
trips["to_code"] = stations.get_indexer(trips["to_id"]).astype("int32")
trips["route"] = trips["from_code"] * 1000 + trips["to_code"]
trips["round_trip"] = (trips["from_code"] == trips["to_code"]).astype("int8")
trips = trips.drop(columns=["from_id", "to_id", "date"])

print(f"\nRaw trips:      {n_raw:,}")
print(f"After cleaning:  {len(trips):,} ({len(trips) / n_raw:.1%})")
print(f"Stations: {len(stations):,} | distinct routes:
{trips['route'].nunique():,} | "
```

```
f"round trips: {trips['round_trip'].mean():.1%}"
trips[["duration_min", "distance_km", "temp_c", "precip_mm"]].describe()
```

```
Trips with no matching weather day:      0
False starts removed:                    274,378
Workshop / test-station moves removed:  1,264
Raw trips:                               10,121,026
After cleaning: 9,837,352 (97.2%)
Stations: 463 | distinct routes: 110,389 | round trips: 3.2%
```

```
Out[4]:
```

	duration_min	distance_km	temp_c	precip_mm
count	9,837,352.000	9,837,352.000	9,837,352.000	9,837,352.000
mean	18.827	2.549	14.772	1.418
std	250.605	2.685	5.283	4.080
min	0.167	0.010	-2.000	0.000
25%	6.400	1.159	11.000	0.000
50%	10.850	1.986	15.700	0.000
75%	18.283	3.342	18.500	0.600
max	279,454.317	3,681.009	24.900	43.100

5. Explore: the outlier problem

This step motivates how the model handles extreme trips, so it is worth looking at carefully.

```
In [5]:
d = trips["duration_min"]
q = d.quantile([0.5, 0.9, 0.99, 0.999, 0.9999])

print(f"mean      {d.mean():8.1f} min")
print(f"median    {d.median():8.1f} min")
for level, value in q.items():
    print(f"p{level * 100:<8g} {value:8.1f} min")
print(f"max      {d.max():8.1f} min  ({d.max() / 60:.1f} hours)")
print()
print(f"Trips over 5 hours: {(d > 300).sum():>8,}  ({(d > 300).mean():.3%} of trips)")
print(f"Trips over 24 hours: {(d > 1440).sum():>8,}  ({(d > 1440).mean():.4%} of trips)")

# How much of each kind of error comes from that tiny tail?
tail = d > 300
sq_error = (d - d.mean()) ** 2
abs_error = (d - d.median()).abs()
log_d = np.log(d)
sq_log_error = (log_d - log_d.mean()) ** 2

print(f"\nTrips over 5 hours are {tail.mean():.3%} of the data, and they contribute:")
print(f" {sq_error[tail].sum() / sq_error.sum():>6.1%} of the squared error around the mean (raw minutes)")
```

```
print(f" {abs_error[tail].sum() / abs_error.sum():>6.1%} of the absolute error
around the median (raw minutes)")
print(f" {sq_log_error[tail].sum() / sq_log_error.sum():>6.1%} of the squared
error in LOG space")
```

```
mean          18.8 min
median        10.8 min
p50           10.8 min
p90           27.6 min
p99           77.9 min
p99.9         1057.2 min
p99.99        4550.0 min
max           279454.3 min (4657.6 hours)
```

```
Trips over 5 hours:    28,374 (0.288% of trips)
Trips over 24 hours:   5,202 (0.0529% of trips)
Trips over 5 hours are 0.288% of the data, and they contribute:
  99.6% of the squared error around the mean (raw minutes)
  34.2% of the absolute error around the median (raw minutes)
  9.0% of the squared error in LOG space
```

This printout is the argument for robustness. Under squared error on raw minutes, the ~0.3% of trips lasting over five hours account for almost **all** of the error being minimised — a model fitted that way is, in effect, fitted to forgotten rentals.

The last line is why we model **log(duration)**. Taking logs compresses a trip of several thousand hours to a few units, so the tail carries far less weight. It still carries much more than its 0.3% share, though. So the log transform and a robust loss do complementary jobs: logs shrink the outliers, and Huber stops the ones that remain from steering the fit. Step 10 tests how much the robust loss still adds.

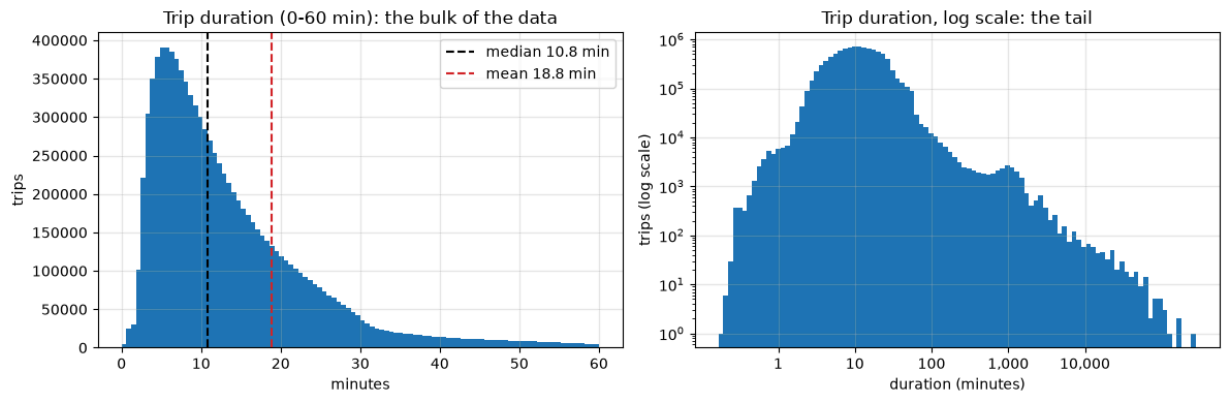
In [6]:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(d, bins=100, range=(0, 60), color="tab:blue")
axes[0].axvline(d.median(), color="black", ls="--", label=f"median
{d.median():.1f} min")
axes[0].axvline(d.mean(), color="tab:red", ls="--", label=f"mean {d.mean():.1f}
min")
axes[0].set_title("Trip duration (0-60 min): the bulk of the data")
axes[0].set_xlabel("minutes")
axes[0].set_ylabel("trips")
axes[0].legend()

axes[1].hist(np.log10(d), bins=100, color="tab:blue")
axes[1].set_title("Trip duration, log scale: the tail")
axes[1].set_xlabel("duration (minutes)")
axes[1].set_xticks([0, 1, 2, 3, 4], ["1", "10", "100", "1,000", "10,000"])
axes[1].set_yscale("log")
axes[1].set_ylabel("trips (log scale)")

plt.tight_layout()
plt.show()
```



6. Explore: duration = distance ÷ speed

Our working assumption is that three things set a trip's duration:

- **length** — a longer route takes longer;
- **time** — the hour and day may bring congestion, or a different kind of trip;
- **weather** — bad weather may slow riders down.

Everything else — a rider stopping for coffee, taking a detour, forgetting to return the bike — is outside our control, so we treat it as noise.

Since duration = distance ÷ speed, we can check *how* each factor acts: does it change how fast people ride, or how far they go? This section uses the distance actually ridden, purely to explore; the model never sees it. Speeds outside 3–35 km/h (mostly forgotten bikes) are left out of this section only.

In [7]:

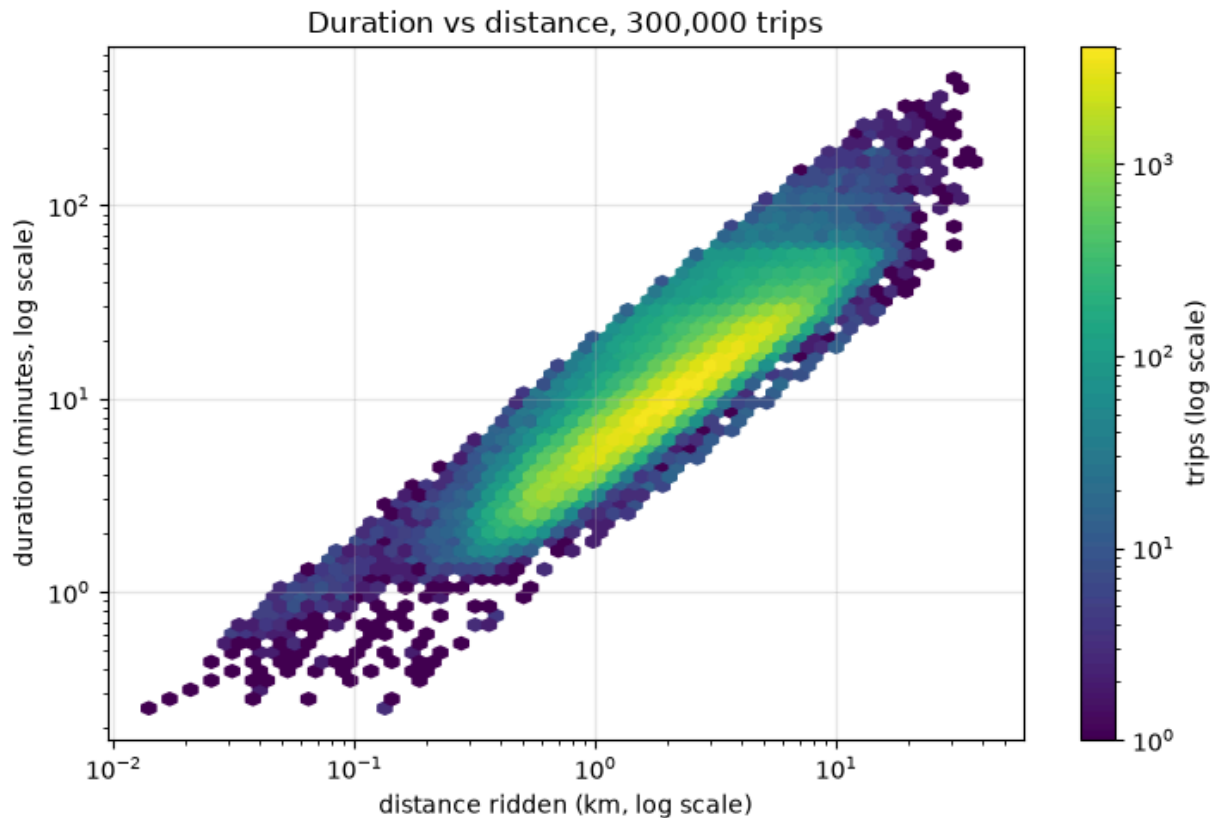
```
ride = trips[["departure", "duration_min", "distance_km", "temp_c",
"precip_mm"]].copy()
ride["speed_kmh"] = ride["distance_km"] / (ride["duration_min"] / 60)
ride = ride[ride["speed_kmh"].between(3, 35)]
print(f"Plausible rides used in this section: {len(ride):,} ({len(ride) /
len(trips):.1%} of trips)")

logs = np.log(ride[["distance_km", "speed_kmh"]])
print("\nlog(duration) = log(distance) - log(speed)")
print(f"  variance of log(distance): {logs['distance_km'].var():.3f}")
print(f"  variance of log(speed):    {logs['speed_kmh'].var():.3f}")
del logs

sample = ride.sample(300_000, random_state=RANDOM_STATE)
fig, ax = plt.subplots(figsize=(7.5, 5))
hb = ax.hexbin(sample["distance_km"], sample["duration_min"], gridsize=60,
               xscale="log", yscale="log", bins="log", mincnt=1, cmap="viridis")
ax.set_xlabel("distance ridden (km, log scale)")
ax.set_ylabel("duration (minutes, log scale)")
ax.set_title("Duration vs distance, 300,000 trips")
fig.colorbar(hb, label="trips (log scale)")
plt.tight_layout()
plt.show()
```

Plausible rides used in this section: 9,597,232 (97.6% of trips)

```
log(duration) = log(distance) - log(speed)
variance of log(distance): 0.536
variance of log(speed): 0.093
```



In [8]:

```
ride["hour"] = ride["departure"].dt.hour
ride["day_type"] = np.where(ride["departure"].dt.dayofweek >= 5, "weekend",
                             "weekday")

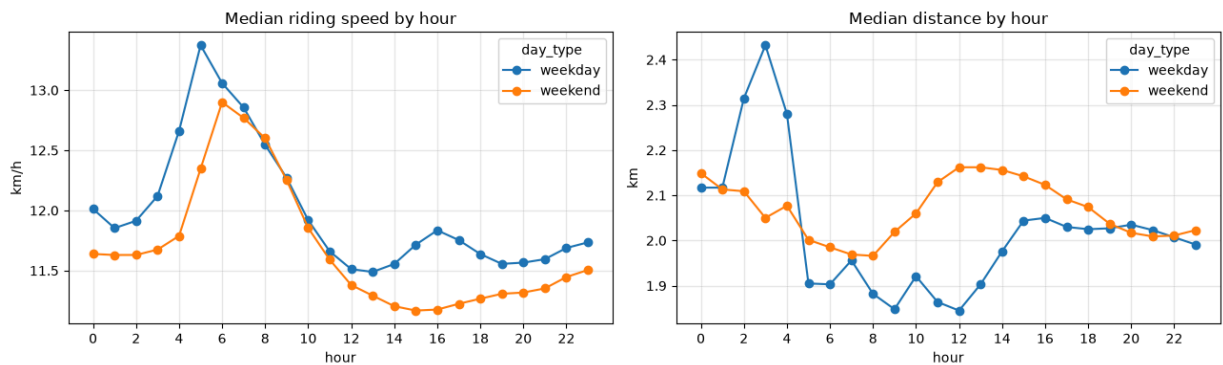
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for ax, column, title, unit in [
    (axes[0], "speed_kmh", "Median riding speed by hour", "km/h"),
    (axes[1], "distance_km", "Median distance by hour", "km"),
]:
    ride.pivot_table(index="hour", columns="day_type", values=column,
                      aggfunc="median").plot(ax=ax, marker="o")
    ax.set_title(title)
    ax.set_ylabel(unit)
    ax.set_xticks(range(0, 24, 2))
plt.tight_layout()
plt.show()

weekday = ride[ride["day_type"] == "weekday"]
periods = {"morning rush 7-9": [7, 8, 9], "midday 11-14": [11, 12, 13, 14],
           "evening rush 16-18": [16, 17, 18], "night 0-5": [0, 1, 2, 3, 4, 5]}
pd.DataFrame({
    label: {
        "speed (km/h)": weekday.loc[weekday["hour"].isin(hours),
        "speed_kmh"].median(),
        "distance (km)": weekday.loc[weekday["hour"].isin(hours),
        "distance_km"].median(),
        "duration (min)": weekday.loc[weekday["hour"].isin(hours),
```

```

"duration_min"].median(),
    }
    for label, hours in periods.items()
}).T.rename_axis("weekdays")

```



Out[8]: **speed (km/h) distance (km) duration (min)**

weekdays			
morning rush 7-9	12.559	1.890	9.383
midday 11-14	11.551	1.899	10.267
evening rush 16-18	11.748	2.035	10.867
night 0-5	12.289	2.150	10.967

In [9]:

```

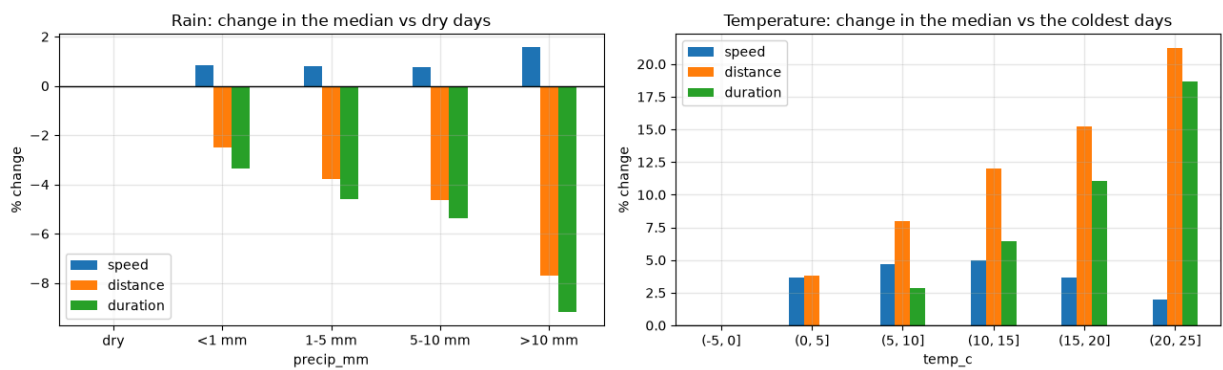
def change_vs_first_group(groups):
    medians = ride.groupby(groups, observed=True).agg(
        speed=("speed_kmh", "median"), distance=("distance_kmh", "median"),
        duration=("duration_min", "median"),
    )
    return 100 * (medians / medians.iloc[0] - 1)

rain = change_vs_first_group(pd.cut(ride["precip_mm"], [-0.01, 0.09, 1, 5, 10,
1e4],
                                labels=["dry", "<1 mm", "1-5 mm", "5-10 mm",
">10 mm"])))
temperature = change_vs_first_group(pd.cut(ride["temp_c"], np.arange(-5, 30, 5)))

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
rain.plot(kind="bar", ax=axes[0])
axes[0].set_title("Rain: change in the median vs dry days")
temperature.plot(kind="bar", ax=axes[1])
axes[1].set_title("Temperature: change in the median vs the coldest days")
for ax in axes:
    ax.axhline(0, color="black", lw=1)
    ax.set_ylabel("% change")
    ax.tick_params(axis="x", rotation=0)
plt.tight_layout()
plt.show()

print("Rain, % change vs dry days:")
print(rain.round(1).to_string())
del ride, weekday, sample # exploration only; frees memory before modelling

```



Rain, % change vs dry days:

	speed	distance	duration
precip_mm			
dry	0.000	0.000	0.000
<1 mm	0.800	-2.500	-3.400
1-5 mm	0.800	-3.800	-4.600
5-10 mm	0.800	-4.600	-5.400
>10 mm	1.600	-7.700	-9.200

Three things stand out, and they sharpen the working assumption:

- Length dominates.** In log terms, distance varies several times more than speed does. Most of the difference between a short and a long trip is simply how far it goes.
- Time changes the *kind* of trip, not the traffic.** If congestion mattered, rush hour would be the slowest time of day. Instead the weekday morning peak is the *fastest*: commuters ride directly and quickly, while midday and afternoon riders are slower. The hour works as a proxy for trip purpose.
- Rain does not slow riders down.** Median speed stays flat — if anything slightly higher — on rainy days, while distance and duration fall. Rain decides *who* rides and how far, not how fast. Temperature behaves similarly: warmer days bring longer trips, with little change in speed.

Point 3 corrects our Stage 1 report, which assumed riders are a little slower in the rain.

This gives a prediction to test. Once route length is in the model, time and weather lose their main channel — which trips happen — so they should add only a small speed adjustment on top. Step 11 measures exactly that.

7. Features and train/validation/test split

Route length. For each trip, its route length is the median distance ridden between the same two stations by *other* trips in *earlier* data. It is a property of the route — the kind of thing a journey planner knows — never the recorded distance of the trip being predicted.

Two details keep this honest:

- Cross-fitting for training rows.** A 2022 training trip gets its route length from 2023 trips only, and vice versa, so no trip ever sees its own distance. Validation trips (2024) use all of 2022–2023; the final test (2025) uses all of 2022–2024.

- **Fallbacks.** A route needs at least 3 earlier trips to have its own length. Otherwise we use the median trip length from the same departure station, and for a station never seen before, the overall median.

Calendar features — hour, weekday, month — are categorical: hour 23 is not "larger" than hour 9 in any useful sense, so one-hot encoding lets the linear model fit each hour freely. Route length enters as a logarithm. Together with the log label, that makes duration a power of length: $\text{duration} \propto \text{length}^b$, where b is learned.

The split is by season, because a random split of a time series leaks information between sets:

Set	Seasons
Training	2022, 2023
Validation	2024
Test	2025

The Huber solver is iterative and slow on millions of rows, so every model is fitted on the **same fixed random sample** of 400,000 training trips. Evaluation always uses every trip in the validation and test seasons. The test season's features are only built in step 13.

In [10]:

```
CATEGORICAL = ["hour", "weekday", "month"]
NUMERIC = ["log_route", "round_trip", "temp_c", "precip_mm"]
FEATURES = CATEGORICAL + NUMERIC
ROUTE_MIN_TRIPS = 3

season = trips["departure"].dt.year

def route_length(df, history):
    # Typical route length for each trip in df, learned only from the trips in
    # `history`.
    # Falls back from the route, to the departure station, to the overall median.
    by_route = history.groupby("route")["distance_m"].agg(["median", "size"])
    by_route = by_route.loc[by_route["size"] >= ROUTE_MIN_TRIPS, "median"]
    by_station = history.groupby("from_code")["distance_m"].median()

    length = df["route"].map(by_route)
    source = pd.Series("route", index=df.index)
    source[length.isna()] = "station"
    length = length.fillna(df["from_code"].map(by_station))
    source[length.isna()] = "overall"
    length = length.fillna(history["distance_m"].median())
    return length.astype("float64"), source

def cross_fitted_route_length(df):
    # Training rows: each season's route lengths come only from the OTHER seasons
    # in df,
    # so no trip ever contributes to its own route length.
    df_season = df["departure"].dt.year
    length = pd.Series(np.nan, index=df.index)
```

```

source = pd.Series("", index=df.index, dtype=object)
for s in sorted(df_season.unique()):
    rows = df_season == s
    s_length, s_source = route_length(df[rows], history=df[~rows])
    length.loc[s_length.index] = s_length
    source.loc[s_source.index] = s_source
return length, source

def make_features(df, length):
    return pd.DataFrame({
        "hour": df["departure"].dt.hour.astype("int16"),
        "weekday": df["departure"].dt.dayofweek.astype("int16"),
        "month": df["departure"].dt.month.astype("int16"),
        "log_route": np.log(length),
        "round_trip": df["round_trip"].astype("int8"),
        "temp_c": df["temp_c"].astype("float64"),
        "precip_mm": df["precip_mm"].astype("float64"),
    }, index=df.index)[FEATURES]

train_rows = trips[season <= 2023]
val_rows = trips[season == 2024]

route_train, source_train = cross_fitted_route_length(train_rows)
route_val, source_val = route_length(val_rows, history=train_rows)

X_train_full, y_train_full = make_features(train_rows, route_train),
train_rows["duration_min"]
X_val, y_val = make_features(val_rows, route_val), val_rows["duration_min"]

sample = X_train_full.sample(TRAIN_SAMPLE, random_state=RANDOM_STATE).index
X_train, y_train = X_train_full.loc[sample], y_train_full.loc[sample]
print(f"training (full) {len(X_train_full):>10,} trips")
print(f"training sample {len(X_train):>10,} trips")
print(f"validation {len(X_val):>10,} trips")
del X_train_full, y_train_full, route_train

coverage = pd.DataFrame({
    "training (cross-fitted)": source_train.value_counts(normalize=True),
    "validation": source_val.value_counts(normalize=True),
}).mul(100).reindex(["route", "station", "overall"]).fillna(0)
print("\nWhere each trip's route length came from (% of trips):")
print(coverage.round(2).to_string())
X_train.head()

```

```

training (full) 4,876,755 trips
training sample 400,000 trips
validation      2,508,741 trips
Where each trip's route length came from (% of trips):
      training (cross-fitted)  validation
route                97.440         96.480
station              2.560          3.520
overall              0.000          0.000

```

Out[10]:

	hour	weekday	month	log_route	round_trip	temp_c	precip_mm
68167	17	5	4	8.685	0	7.900	0.000
186345	8	1	5	7.343	0	11.600	3.600
2307348	9	4	10	6.714	0	10.300	3.400
3835782	17	0	7	7.981	0	15.900	0.000
923465	18	0	6	8.058	0	14.100	0.000

8. Three models and two reference points

Each pair of models differs in exactly one thing, so each comparison answers one question:

Model	Hypothesis space	Loss
Linear, squared error	linear	squared error
Linear, Huber	linear	Huber
Boosted trees, absolute error	non-linear	absolute error

- **Squared error vs Huber** — same hypothesis space, different loss: *does the robust loss still matter once the label is logged?*
- **Linear (Huber) vs boosted trees** — both robust, different hypothesis space: *are there interactions, such as "rain matters more on long routes", that a linear model misses?*

`HuberRegressor` behaves like squared error for small residuals and like absolute error beyond a threshold ($\epsilon = 1.35$ scaled residuals). The trees use absolute error because scikit-learn's histogram-based booster has no Huber option; the two robust losses behaved almost identically in our earlier comparison. All three are fitted on $\log(\text{duration})$, and their predictions are converted back to minutes.

Two reference points are not models, but anchor the numbers:

- **Constant median** — the same duration for every trip.
- **Physics** — route length $\times$ one typical pace (minutes per km), estimated from training data. This is our working assumption with no machine learning at all. If the models barely beat it, the learning is only worth the speed adjustments on top of arithmetic.

Metrics:

- **MAE** — mean absolute error, in minutes.
- **MedAE** — median absolute error: the typical trip's error, unaffected by forgotten rentals.
- **Within 2 min** — the share of trips predicted to within two minutes, the most direct measure of usefulness to a rider.

The selection rule is fixed before seeing any results: **the model with the lowest validation MAE is chosen** for the final test. RMSE and R^2 are not used — both are squared-error quantities, which on this label mostly measure forgotten rentals.

In [11]:

```
class ConstantMedian:
    # Reference point: the same prediction for every trip.
    def fit(self, X, y):
        self.value_ = float(np.median(y))
        return self

    def predict(self, X):
        return np.full(len(X), self.value_)

class PhysicsBaseline:
    # Reference point: route Length times one typical pace. No machine Learning.
    def fit(self, X, y):
        route_km = np.exp(X["log_route"]) / 1000
        self.min_per_km_ = float(np.median(y / route_km))
        return self

    def predict(self, X):
        return np.exp(X["log_route"]).to_numpy() / 1000 * self.min_per_km_

def linear_model(regressor, features=FEATURES):
    cats = [f for f in features if f in CATEGORICAL]
    nums = [f for f in features if f not in CATEGORICAL]
    parts = []
    if cats:
        parts.append(("cat", OneHotEncoder(handle_unknown="ignore"), cats))
    if nums:
        parts.append(("num", StandardScaler(), nums))
    return make_pipeline(ColumnTransformer(parts), regressor)

def on_log_scale(model):
    # Fit on Log(duration); predictions come back in minutes.
    return TransformedTargetRegressor(regressor=model, func=np.log,
    inverse_func=np.exp)

def huber():
    return HuberRegressor(epsilon=1.35, alpha=1e-4, max_iter=500)

def build_models():
    return {
        "Linear, squared error": on_log_scale(linear_model(LinearRegression())),
        "Linear, Huber": on_log_scale(linear_model(huber())),
        "Boosted trees, absolute error":
on_log_scale(HistGradientBoostingRegressor(
        loss="absolute_error", categorical_features=CATEGORICAL,
        random_state=RANDOM_STATE,
    )),
    }

def build_references():
    return {
        "Constant: median": ConstantMedian(),
    }
```

```

        "Physics: route length x typical pace": PhysicsBaseline(),
    }

def evaluate(y_true, y_pred):
    err = np.abs(np.asarray(y_true, dtype="float64") - np.asarray(y_pred,
dtype="float64"))
    return {"MAE": err.mean(), "MedAE": np.median(err), "within 2 min %": 100 *
(err <= 2).mean()}

def report_solver_warnings(caught):
    messages = sorted({str(w.message).splitlines()[0] for w in caught})
    print("Solver warnings:", "none" if not messages else "")
    for m in messages:
        print("  -", m)

```

In [12]:

```

references, models = build_references(), build_models()
val_pred, rows = {}, []

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    for name, model in (**references, **models).items():
        model.fit(X_train, y_train)
        val_pred[name] = model.predict(X_val)
        rows.append({"model": name, **evaluate(y_val, val_pred[name])})
report_solver_warnings(caught)

results = pd.DataFrame(rows).set_index("model")
baseline_medae = results.loc["Constant: median", "MedAE"]
results["MedAE gain vs constant %"] = 100 * (baseline_medae - results["MedAE"]) /
baseline_medae

physics = references["Physics: route length x typical pace"]
print(f"Physics reference: {physics.min_per_km_:.2f} min per km of route "
      f"(= {60 / physics.min_per_km_:.2f} km/h)")

best_name = results.loc[list(models), "MAE"].idxmin()
print(f"Selected by the pre-declared rule (lowest validation MAE of the three
models): {best_name}")
results

```

Solver warnings: none

Physics reference: 5.23 min per km of route (= 11.48 km/h)

Selected by the pre-declared rule (lowest validation MAE of the three models):

Linear, Huber

Out[12]:

	MAE	MedAE	within 2 min %	MedAE gain vs constant %
model				
Constant: median	13.978	5.300	18.984	0.000
Physics: route length x typical pace	11.219	2.059	49.131	61.145
Linear, squared error	11.002	2.205	46.099	58.403
Linear, Huber	10.909	1.857	52.461	64.956
Boosted trees, absolute error	10.955	1.853	52.456	65.041

In [13]:

```
# Training error: every predictor scored on the 400,000 training trips it was
fitted on,
# next to its validation error. A large gap between the two would indicate
overfitting.
training = pd.DataFrame(
    [{"model": name, **evaluate(y_train, model.predict(X_train))} for name, model
in {**references, **models}.items()]
).set_index("model")
metrics = ["MAE", "MedAE", "within 2 min %"]
pd.concat({"training": training[metrics], "validation": results[metrics]}, axis=1)
```

Out[13]:

	training			validation		
	MAE	MedAE	within 2 min %	MAE	MedAE	within 2 min %
model						
Constant: median	10.404	4.883	20.251	13.978	5.300	18.984
Physics: route length x typical pace	7.921	1.755	54.063	11.219	2.059	49.131
Linear, squared error	7.736	1.983	50.362	11.002	2.205	46.099
Linear, Huber	7.622	1.617	57.696	10.909	1.857	52.461
Boosted trees, absolute error	7.570	1.576	58.328	10.955	1.853	52.456

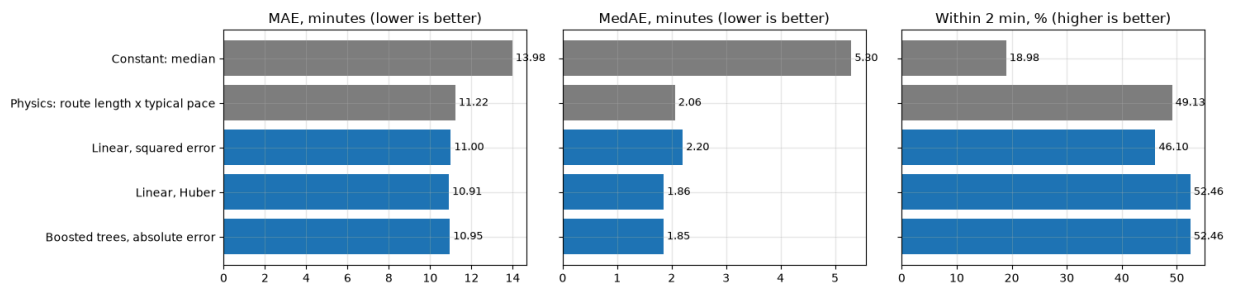
In [14]:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.6), sharey=True)
colors = ["tab:gray" if name in references else "tab:blue" for name in
results.index]
for ax, column, title in zip(
    axes,
    ["MAE", "MedAE", "within 2 min %"],
    ["MAE, minutes (lower is better)", "MedAE, minutes (lower is better)", "Within
2 min, % (higher is better)"],
):
    ax.barh(results.index, results[column], color=colors)
    for y, v in enumerate(results[column]):
```

```

ax.text(v, y, f" {v:.2f}", va="center", fontsize=9)
ax.set_title(title)
axes[0].invert_yaxis()
plt.tight_layout()
plt.show()

```



Three readings of this table:

- 1. Route length does most of the work.** The physics reference — route length times a single pace of 5.23 min/km (11.48 km/h), no learning at all — cuts the typical error from 5.30 to 2.06 minutes. That is 61 of the 65 percentage points of improvement any model achieves.
- 2. The loss still matters with a logged label.** On typical trips, the squared-error model is *worse than the physics reference*: MedAE 2.205 vs 2.059 minutes, and 46.1% vs 49.1% of trips within 2 minutes. Huber gets 52.5%. The remaining tail pulls the squared-error fit upwards (step 10 shows its predictions run about half a minute high), so it overestimates the ordinary trip.
- 3. Linear and non-linear are indistinguishable.** Huber and the boosted trees differ by 0.05 minutes of MAE and 0.004 minutes of MedAE, with identical within-2-minute rates. Even with route length available to interact with, the trees find nothing the linear model misses.

Training vs validation. Every predictor has a lower error on its training trips than on 2024, including the constant and the physics reference, which fit at most one number (the constant's MedAE is 4.88 on training against 5.30 on validation). So the gap mostly reflects differences between seasons, not overfitting. The boosted trees reach the lowest training error (MedAE 1.58) but have no validation advantage over the linear model. That is a mild sign that their extra flexibility fits the training data rather than the problem.

By the pre-declared rule (lowest validation MAE), **Linear, Huber** is selected for the final test.

9. Are the differences real? A day-block bootstrap

Differences between models can be a fraction of a minute, so we need to know whether they would survive a different season. We resample the 2024 validation season **by whole days** 2,000 times — trips on the same day share weather and events, so they are not independent — and recompute every model's errors on each resample.

Comparing two models on the *same* resampled days gives a 95% interval for their **paired difference**. If the interval excludes zero, the difference is clear.

In [15]:

```
COMPARISONS = [
    # (question, first model, second model, short name of first, short name of
    second)
    ("loss", "Linear, Huber", "Linear, squared error", "Huber", "squared"),
    ("hypothesis space", "Boosted trees, absolute error", "Linear, Huber",
    "trees", "linear"),
    ("learning vs physics", best_name, "Physics: route length x typical pace",
    "model", "physics"),
]

def verdict(low, high, first, second, lower_is_better):
    if low <= 0 <= high:
        return "no clear difference"
    first_wins = high < 0 if lower_is_better else low > 0
    return f"{first if first_wins else second} better"

def paired_bootstrap(y_true, predictions, dates, n_resamples=2000,
seed=RANDOM_STATE):
    # 95% intervals for paired differences, resampling whole days.
    day_index, days = pd.factorize(dates)
    n_days = len(days)
    y_true = np.asarray(y_true, dtype="float64")
    trips_per_day = np.bincount(day_index, minlength=n_days)

    def per_day(values):
        return np.bincount(day_index, weights=values, minlength=n_days)

    abs_err = {name: per_day(np.abs(y_true - p)) for name, p in
predictions.items()}
    within = {name: per_day((np.abs(y_true - p) <= 2).astype("float64")) for name,
p in predictions.items()}

    draws = np.random.default_rng(seed).integers(0, n_days, size=(n_resamples,
n_days))
    boot_trips = trips_per_day[draws].sum(axis=1)

    def boot(by_day):
        return by_day[draws].sum(axis=1) / boot_trips

    def overall(by_day):
        return by_day.sum() / trips_per_day.sum()

    rows = []
    for question, a, b, short_a, short_b in COMPARISONS:
        mae_lo, mae_hi = np.percentile(boot(abs_err[a]) - boot(abs_err[b]), [2.5,
97.5])
        w_lo, w_hi = np.percentile(100 * (boot(within[a]) - boot(within[b])),
[2.5, 97.5])
        rows.append({
            "question": question,
            "comparison": f"{short_a} - {short_b}",
            "MAE diff (min)": overall(abs_err[a]) - overall(abs_err[b]),
            "MAE 95% CI": f"[{mae_lo:+.3f}, {mae_hi:+.3f}]",
            "MAE verdict": verdict(mae_lo, mae_hi, short_a, short_b,
lower_is_better=True),
            "within-2 diff (pp)": 100 * (overall(within[a]) - overall(within[b])),
```



```

        "within-2 95% CI": f"[{w_lo:+.2f}, {w_hi:+.2f}]",
        "within-2 verdict": verdict(w_lo, w_hi, short_a, short_b,
lower_is_better=False),
    })
    print(f"{n_days} days, {n_resamples:,} day-level resamples")
    return pd.DataFrame(rows).set_index("question")

paired_bootstrap(y_val, val_pred, val_rows["departure"].dt.normalize())

```

214 days, 2,000 day-level resamples

Out[15]:

	comparison	MAE diff (min)	MAE 95% CI	MAE verdict	within-2 diff (pp)	within-2 95% CI	within-2 verdict
question							
loss	Huber - squared	-0.093	[-0.098, -0.088]	Huber better	6.362	[+6.14, +6.57]	Huber better
hypothesis space	trees - linear	0.046	[+0.043, +0.049]	linear better	-0.005	[-0.07, +0.06]	no clear difference
learning vs physics	model - physics	-0.310	[-0.317, -0.302]	model better	3.330	[+3.12, +3.54]	model better

- **Loss — Huber is clearly better.** Both intervals are far from zero: 0.093 minutes lower MAE, and **6.4 percentage points more trips within 2 minutes**. The second number is the one that matters to a rider: with Huber, roughly one extra trip in sixteen lands within two minutes.
- **Hypothesis space — no practical difference.** On MAE the linear model is "clearly" better, but by 0.046 minutes — under three seconds. With 2.5 million trips, even a trivial difference produces a tight interval. On within-2-minutes there is no detectable difference at all. *Statistically detectable is not the same as practically meaningful.*
- **Learning vs physics — real but modest.** The model beats pure arithmetic by 0.31 minutes of MAE and 3.3 percentage points within 2 minutes.

These intervals describe uncertainty *within* the 2024 season. They cannot capture year-to-year change; that is what the 2025 test in step 13 checks.

10. Does the loss still matter? An outlier-sensitivity experiment

Logging the label already shrinks the forgotten rentals, so the question is whether the robust loss still adds anything. We refit each model with the longest 0.1% of training trips removed, and measure how far its predictions move on 200,000 validation trips.

Any refit moves a model a little, even with nothing special removed — boosted trees, for instance, hold out a random internal split for early stopping. So we also refit with the **same**

number of trips removed at random, as a control. The outlier effect is the difference between the two shifts.

In [16]:

```
cutoff = y_train.quantile(0.999)
longest = y_train > cutoff
n_removed = int(longest.sum())
random_drop = y_train.sample(n_removed, random_state=RANDOM_STATE + 1).index
keep_trimmed = ~longest
keep_random = ~y_train.index.isin(random_drop)
probe = X_val.sample(200_000, random_state=RANDOM_STATE)
print(f"Removed {n_removed:,} training trips longer than {cutoff:,.0f} min
({n_removed / len(y_train):.2%}); "
      f"control: {n_removed:,} trips removed at random")

def mean_shift_pct(new, reference):
    return 100 * (np.abs(new - reference) / reference).mean()

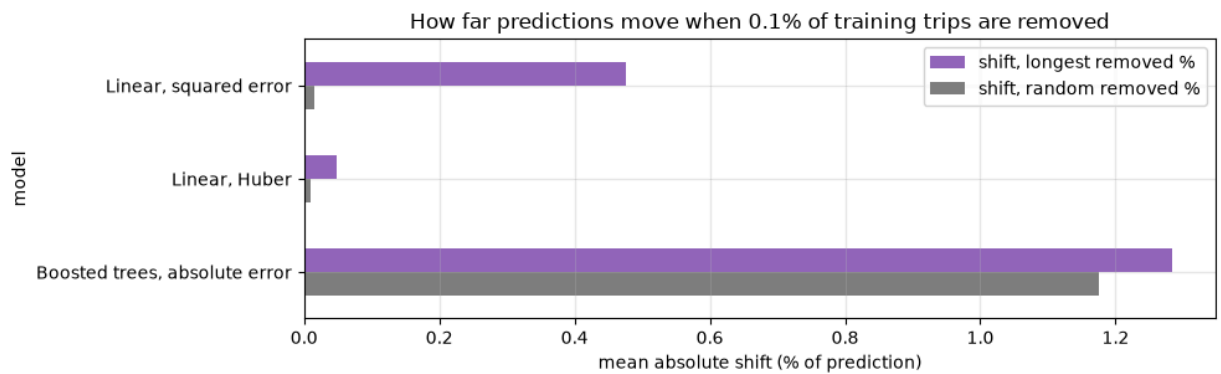
rows = []
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    for name in models:
        full = models[name].predict(probe)
        trimmed = build_models()[name].fit(X_train[keep_trimmed],
y_train[keep_trimmed]).predict(probe)
        control = build_models()[name].fit(X_train[keep_random],
y_train[keep_random]).predict(probe)
        rows.append({
            "model": name,
            "mean prediction (min)": full.mean(),
            "shift, longest removed %": mean_shift_pct(trimmed, full),
            "shift, random removed %": mean_shift_pct(control, full),
        })
report_solver_warnings(caught)

sensitivity = pd.DataFrame(rows).set_index("model")
sensitivity["outlier effect (pp)"] = sensitivity["shift, longest removed %"] -
sensitivity["shift, random removed %"]

ax = sensitivity[["shift, longest removed %", "shift, random removed %"]].plot(
    kind="barh", figsize=(10, 3.2), color=["tab:purple", "tab:gray"])
ax.set_title("How far predictions move when 0.1% of training trips are removed")
ax.set_xlabel("mean absolute shift (% of prediction)")
ax.invert_yaxis()
plt.tight_layout()
plt.show()
sensitivity
```

Removed 400 training trips longer than 1,016 min (0.10%); control: 400 trips removed at random

Solver warnings: none



Out[16]:

	mean prediction (min)	shift, longest removed %	shift, random removed %	outlier effect (pp)
model				
Linear, squared error	12.034	0.476	0.015	0.461
Linear, Huber	11.486	0.048	0.009	0.039
Boosted trees, absolute error	11.289	1.284	1.175	0.109

The control changes how this table should be read:

- **Boosted trees** move 1.28% when the longest trips are removed, but 1.18% when the same number of *random* trips is removed. Almost all of that movement is ordinary refit variance, not outlier influence. The true outlier effect is 0.11 percentage points.
- **Squared error** moves 0.48% against a control of 0.015%. That outlier effect (0.46 points) is **about 12 times Huber's** (0.04 points).

Compared with the earlier raw-minute version of this experiment, where squared-error predictions moved by 17–23%, the log transform has removed most of the instability, and Huber removes most of what is left.

In log space, the larger cost of squared error is not instability but **bias**. Its average prediction is 12.03 minutes, against 11.49 for Huber — about half a minute too high on ordinary trips. That is why it loses 6 percentage points on trips within 2 minutes (step 9). The log transform and Huber do complementary jobs: logs handle stability, and Huber removes the bias on typical trips.

11. Testing the three-factor assumption: feature ablation

Here the working assumption from step 6 is tested directly. We refit the Huber model with each combination of factors and measure what each one adds:

- **Length** — route length plus the round-trip flag (both describe the route);
- **Time** — hour, weekday, month;
- **Weather** — temperature and precipitation.

The first row repeats the Stage 1 feature set — time and weather without the route — for comparison.

In [17]:

```
ABLATION = {
    "Time + weather (Stage 1 features)": ["hour", "weekday", "month", "temp_c",
    "precip_mm"],
    "Length": ["log_route", "round_trip"],
    "Length + time": ["log_route", "round_trip", "hour", "weekday", "month"],
    "Length + weather": ["log_route", "round_trip", "temp_c", "precip_mm"],
    "Length + time + weather": FEATURES,
}

rows = [{"features": "None (constant median)", **evaluate(y_val,
val_pred["Constant: median"])}]
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    for label, features in ABLATION.items():
        if features == FEATURES:
            pred = val_pred["Linear, Huber"] # the full model, already fitted in
step 8
        else:
            pred = on_log_scale(linear_model(huber(), features)).fit(X_train,
y_train).predict(X_val)
        rows.append({"features": label, **evaluate(y_val, pred)})
report_solver_warnings(caught)

ablation = pd.DataFrame(rows).set_index("features")
none_medae = ablation.loc["None (constant median)", "MedAE"]
full_medae = ablation.loc["Length + time + weather", "MedAE"]
ablation["share of total MedAE gain %"] = 100 * (none_medae - ablation["MedAE"]) /
(none_medae - full_medae)
ablation
```

Solver warnings: none

Out[17]:

	MAE	MedAE	within 2 min %	share of total MedAE gain %
features				
None (constant median)	13.978	5.300	18.984	0.000
Time + weather (Stage 1 features)	13.932	5.250	19.104	1.458
Length	10.950	1.865	52.326	99.778
Length + time	10.913	1.850	52.587	100.225
Length + weather	10.945	1.875	52.165	99.501
Length + time + weather	10.909	1.857	52.461	100.000

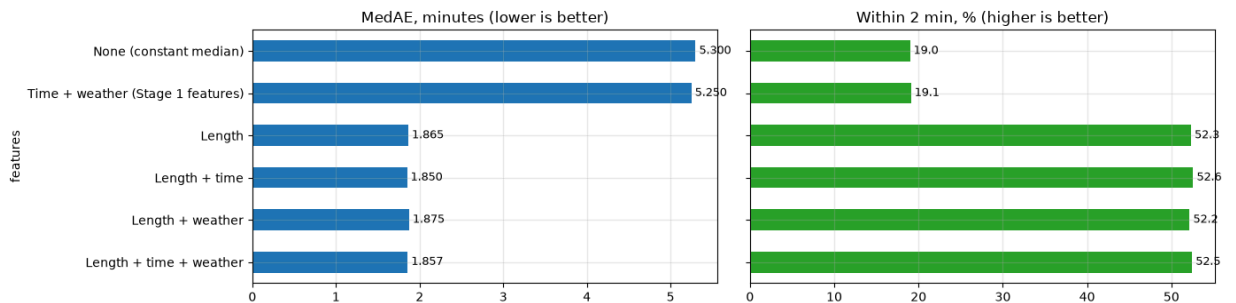
In [18]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 3.6), sharey=True)
ablation["MedAE"].plot(kind="barh", ax=axes[0], color="tab:blue")
axes[0].set_title("MedAE, minutes (lower is better)")
ablation["within 2 min %"].plot(kind="barh", ax=axes[1], color="tab:green")
```

```

axes[1].set_title("Within 2 min, % (higher is better)")
axes[0].invert_yaxis()
for ax, column, fmt in [(axes[0], "MedAE", "{:.3f}"), (axes[1], "within 2 min %", "{:.1f}")] :
    for y, v in enumerate(ablation[column]):
        ax.text(v, y, " " + fmt.format(v), va="center", fontsize=9)
plt.tight_layout()
plt.show()

```



This scores the three-factor assumption from step 6:

- **Length: confirmed, and dominant.** Route length alone delivers 99.8% of the total improvement: MedAE falls from 5.30 to 1.865 minutes, and trips within 2 minutes rise from 19.0% to 52.3%.
- **Time: confirmed, but small.** On top of length, the calendar features improve all three metrics consistently: MedAE 1.865 → 1.850, +0.26 percentage points within 2 minutes, −0.037 minutes MAE.
- **Weather: not supported.** Adding weather to length slightly *worsens* the typical-trip metrics (MedAE 1.865 → 1.875, −0.16 points within 2 minutes) and improves MAE by a negligible 0.005 minutes. The signs are mixed and the sizes tiny: at daily resolution, there is no evidence that weather helps.

The first row repeats the Stage 1 feature set: time and weather without the route deliver just **1.5%** of the improvement. That is the quantitative case for the Stage 2 reformulation.

By MedAE, the best set is length + time, not the full model. We keep the full model because the feature set and the selection rule were fixed before looking at these results; dropping weather now would mean choosing features on the same data we report.

12. What the model learned

Because the model is linear in log space, each coefficient converts into a percentage change in duration. These come from the full Huber model fitted in step 8.

In [19]:

```

fitted = models["Linear, Huber"].regressor_
pre = fitted.named_steps["columntransformer"]
coef = pd.Series(fitted.named_steps["huberregressor"].coef_,
index=pre.get_feature_names_out())
numeric_names = [f for f in FEATURES if f not in CATEGORICAL]
per_unit = coef[[f"num_{n}" for n in numeric_names]].to_numpy() /
pre.named_transformers_["num"].scale_

```

```

per_unit = pd.Series(per_unit, index=numeric_names)

def pct(log_effect):
    return 100 * (np.exp(log_effect) - 1)

b = per_unit["log_route"]
effects = pd.Series({
    "route length exponent b (duration ~ length^b)": b,
    "doubling the route length: % longer trip": pct(b * np.log(2)),
    "round trip vs one-way: %": pct(per_unit["round_trip"]),
    "+1 C temperature: %": pct(per_unit["temp_c"]),
    "+1 mm/day rain: %": pct(per_unit["precip_mm"]),
    "+10 mm/day rain: %": pct(10 * per_unit["precip_mm"]),
})
print(effects.round(3).to_string())

hours = coef[[n for n in coef.index if n.startswith("cat__hour_")]]
hours.index = [int(n.rsplit("_", 1)[1]) for n in hours.index]
hours = pct(hours.sort_index() - hours.mean())

weekdays = coef[[n for n in coef.index if n.startswith("cat__weekday_")]]
weekdays.index = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
weekdays = pct(weekdays - weekdays.mean())

print("\nHour of day, % change in duration vs the daily average (same route):")
for start in (0, 12):
    print(" " + " ".join(f"{h:02d}h {hours[h]:+5.1f}" for h in range(start,
start + 12)))
print("Weekday, % change: " + " ".join(f"{d} {v:+.1f}" for d, v in
weekdays.items()))

fig, axes = plt.subplots(1, 2, figsize=(14, 3.8), gridspec_kw={"width_ratios": [3,
1]})
axes[0].bar(hours.index, hours.values, color=np.where(hours.values < 0,
"tab:green", "tab:red"))
axes[0].set_title("Hour of day: % change in duration vs the daily average (same
route)")
axes[0].set_xlabel("hour")
axes[0].set_xticks(range(0, 24, 2))
axes[1].bar(weekdays.index, weekdays.values, color=np.where(weekdays.values < 0,
"tab:green", "tab:red"))
axes[1].set_title("Weekday: % change")
for ax in axes:
    ax.axhline(0, color="black", lw=1)
    ax.set_ylabel("%")
plt.tight_layout()
plt.show()

```

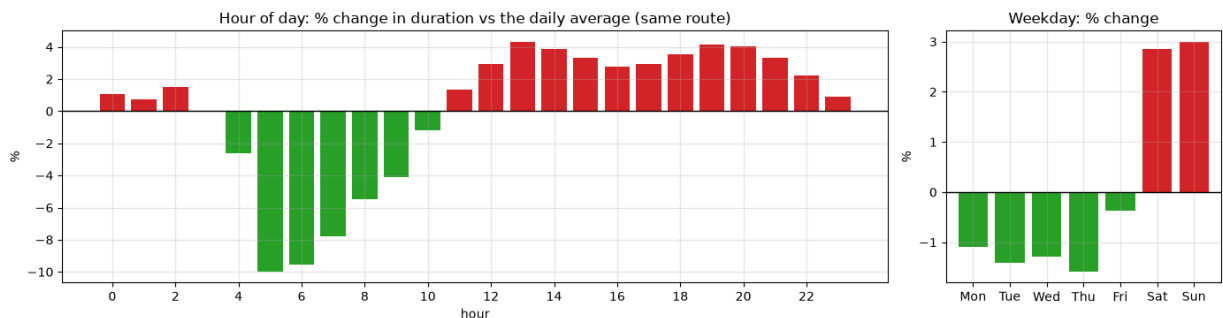
route length exponent b (duration $\sim \text{length}^b$)	0.847
doubling the route length: % longer trip	79.926
round trip vs one-way: %	104.982
+1 C temperature: %	0.236
+1 mm/day rain: %	-0.120
+10 mm/day rain: %	-1.190

Hour of day, % change in duration vs the daily average (same route):

00h +1.1 01h +0.7 02h +1.5 03h -0.1 04h -2.6 05h -10.0 06h -9.5 07h -7.8 08h -5.5 09h -4.1 10h -1.2 11h +1.4

12h +2.9 13h +4.3 14h +3.9 15h +3.3 16h +2.8 17h +2.9 18h +3.6 19h +4.1 20h +4.0 21h +3.3 22h +2.2 23h +0.9

Weekday, % change: Mon -1.1 Tue -1.4 Wed -1.3 Thu -1.6 Fri -0.4 Sat +2.9 Sun +3.0



Every effect below holds the route fixed, so it describes *speed* on the same journey:

- **Route length: duration $\propto \text{length}^{0.85}$.** Doubling the route makes a trip 80% longer, not 100%. Longer trips are ridden at a higher average speed — fewer junctions and stops per kilometre.
- **Round trips take about twice as long** (+105%) as one-way trips of the same typical length. A loop back to the starting station is a leisure ride, with stops.
- **Hour of day spans about 14 percentage points.** Trips are fastest at dawn and in the morning commute (−10% at 05:00, −5.5% at 08:00) and slowest in the early afternoon and evening (+4.3% at 13:00, +4.1% at 19:00). This matches step 6: the hour reflects *trip purpose*, not congestion.
- **Weekends are slower:** Saturday and Sunday about +3%, Monday to Thursday about −1% to −1.6%, so roughly 4–5% between them.
- **Temperature:** +0.24% per °C, so a day 20 °C warmer means trips about 5% longer — more leisurely riding.
- **Rain:** −0.12% per mm, so 10 mm of rain makes the same route about 1.2% *faster*. This is the opposite of our Stage 1 assumption, and consistent with step 6: riders hurry in the rain.

These effects are real and easy to interpret, but they are small next to the differences between individual riders. That is why time and weather add so little to accuracy (step 11).

13. Final evaluation on the 2025 test season

The model choice was made on 2024. Now every model is refitted on 2022–2024 — with route lengths relearned from those seasons, cross-fitted as before — and evaluated **once** on

the untouched 2025 season. All three are reported for transparency, with the selected one marked.

2025 also opened new stations, which have no history; their trips fall back to the station or overall length.

In [20]:

```
trainval_rows = trips[season <= 2024]
test_rows = trips[season == 2025]

route_tv, _ = cross_fitted_route_length(trainval_rows)
route_test, source_test = route_length(test_rows, history=trainval_rows)

X_tv_full, y_tv_full = make_features(trainval_rows, route_tv),
trainval_rows["duration_min"]
X_test, y_test = make_features(test_rows, route_test), test_rows["duration_min"]
sample_tv = X_tv_full.sample(TRAIN_SAMPLE, random_state=RANDOM_STATE).index
X_tv, y_tv = X_tv_full.loc[sample_tv], y_tv_full.loc[sample_tv]
del X_tv_full, y_tv_full, route_tv

print(f"Test season 2025: {len(X_test):,} trips")
print("Where each test trip's route length came from (% of trips):")
print((100 * source_test.value_counts(normalize=True)).reindex(["route",
"station", "overall"]).fillna(0).round(2).to_string())

test_pred, rows = {}, []
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    for name, model in {**build_references(), **build_models()}.items():
        model.fit(X_tv, y_tv)
        test_pred[name] = model.predict(X_test)
        rows.append({"model": name, **evaluate(y_test, test_pred[name])})
report_solver_warnings(caught)

test_results = pd.DataFrame(rows).set_index("model")
test_results["selected on 2024"] = np.where(test_results.index == best_name, "<-
selected", "")
test_results
```

Test season 2025: 2,451,856 trips

Where each test trip's route length came from (% of trips):

route 97.400

station 2.240

overall 0.350

Solver warnings: none

Out[20]:

	MAE	MedAE	within 2 min %	selected on 2024
model				
Constant: median	14.892	5.433	18.610	
Physics: route length x typical pace	11.949	2.140	48.022	
Linear, squared error	11.704	2.353	43.345	
Linear, Huber	11.548	1.911	51.526	<- selected
Boosted trees, absolute error	11.599	1.890	51.813	

In [21]:

```
# The same paired comparisons as step 9, now on the untouched test season.
paired_bootstrap(y_test, test_pred, test_rows["departure"].dt.normalize())
```

214 days, 2,000 day-level resamples

Out[21]:

		MAE diff (min)	MAE 95% CI	MAE verdict	within-2 diff (pp)	within-2 95% CI	within-2 verdict
question							
loss	Huber - squared	-0.156	[-0.163, -0.150]	Huber better	8.182	[+7.88, +8.47]	Huber better
hypothesis space	trees - linear	0.051	[+0.046, +0.057]	linear better	0.286	[+0.20, +0.37]	trees better
learning vs physics	model - physics	-0.401	[-0.409, -0.391]	model better	3.504	[+3.23, +3.75]	model better

In [22]:

```
chosen = pd.DataFrame({
    "actual": y_test.to_numpy(),
    "predicted": test_pred[best_name],
    "route_km": np.exp(X_test["log_route"].to_numpy()) / 1000,
    "hour": X_test["hour"].to_numpy(),
})
chosen["route band"] = pd.qcut(chosen["route_km"], 10, duplicates="drop")

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
by_route = chosen.groupby("route band", observed=True)[["actual",
"predicted"]].median()
axes[0].plot(range(len(by_route)), by_route["actual"], marker="o", label="actual
median")
axes[0].plot(range(len(by_route)), by_route["predicted"], marker="s", ls="--",
label="predicted median")
axes[0].set_xticks(range(len(by_route)))
axes[0].set_xticklabels([f"{iv.mid:.1f}" for iv in by_route.index], fontsize=8)
axes[0].set_xlabel("route length band (km, midpoint)")
axes[0].set_ylabel("minutes")
axes[0].set_title("2025: duration by route length")
axes[0].legend()

# Within one hour, short and long routes are mixed. Because trips scatter further
```

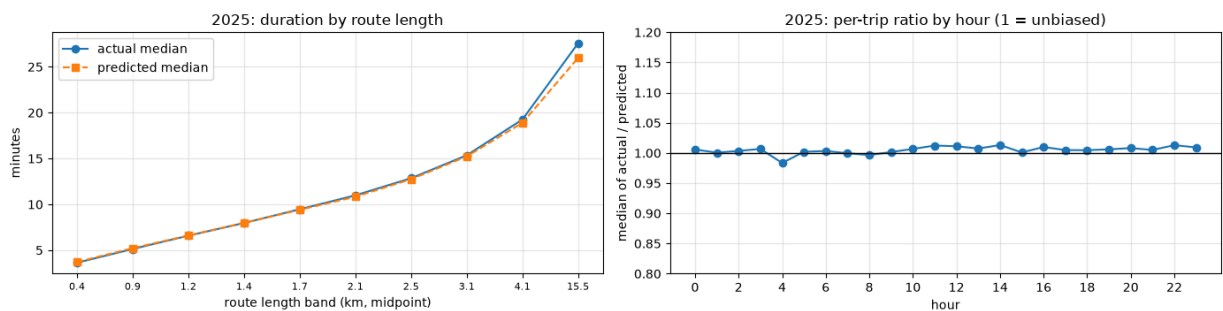
```

above
# their prediction than below it, the median of a MIX of routes sits above the
median of
# their predictions even when each prediction is unbiased. The per-trip ratio
avoids that.
ratio = chosen["actual"] / chosen["predicted"]
ratio_by_hour = ratio.groupby(chosen["hour"]).median()
axes[1].plot(ratio_by_hour.index, ratio_by_hour.values, marker="o")
axes[1].axhline(1, color="black", lw=1)
axes[1].set_ylim(0.8, 1.2)
axes[1].set_xlabel("hour")
axes[1].set_xticks(range(0, 24, 2))
axes[1].set_ylabel("median of actual / predicted")
axes[1].set_title("2025: per-trip ratio by hour (1 = unbiased)")
fig.suptitle(f"Selected model: {best_name}", y=1.02)
plt.tight_layout()
plt.show()

print(f"Median actual {chosen['actual'].median():.2f} min | median predicted
{chosen['predicted'].median():.2f} min")
print(f"Median of actual / predicted, all 2025 trips: {ratio.median():.3f}")
print(f"Hourly median ratio ranges from {ratio_by_hour.min():.3f} to
{ratio_by_hour.max():.3f}")

```

Selected model: Linear, Huber



Median actual 11.52 min | median predicted 10.41 min
Median of actual / predicted, all 2025 trips: 1.006
Hourly median ratio ranges from 0.984 to 1.013

The choice holds up on a season the model has never seen:

- **Linear, Huber: MAE 11.55, MedAE 1.91 min, 51.5% of trips within 2 minutes** — close to its 2024 validation results (1.86 min, 52.5%), so the model generalises to a new season.
- **Against the constant baseline**, the typical error falls from 5.43 to 1.91 minutes (–65%), and trips within 2 minutes rise from 18.6% to 51.5%.
- **The ranking is the same as in 2024.** Huber beats squared error clearly — 0.156 minutes of MAE and **8.2 percentage points within 2 minutes**, a wider gap than in 2024. It also beats the physics reference by 0.40 minutes of MAE and 3.5 points.
- **Trees vs linear stays split by metric.** Linear is 0.05 minutes better on MAE; the trees are 0.29 points better within 2 minutes (MedAE 1.890 vs 1.911). Both differences are statistically detectable and practically negligible.
- **No systematic bias.** Across all 2025 trips, the median of actual ÷ predicted is 1.006, and hour by hour it stays between 0.98 and 1.01 (right panel). Comparing the *median actual* time (11.5 min) with the *median prediction* (10.4 min) would suggest the model runs a

minute short. That is an aggregation artifact: trips scatter further above their prediction than below it, so a mix of short and long routes has a higher median than its predictions do, even when every prediction is right. The per-trip ratio is the honest check.

- **MAE stays around 11.5 minutes while MedAE is about 1.9.** Forgotten rentals still dominate the *average* error, and no feature can predict them.

97.4% of 2025 trips had a known route length. 2.2% fell back to their departure station, and 0.35% — trips from stations that opened in 2025 — to the overall median.

14. Conclusions

The answer to the project question. Given the route, the time and the weather, the selected model — linear regression on $\log(\text{duration})$ with Huber loss — predicts the duration of a 2025 trip with a typical error of **1.9 minutes**, and gets **half of all trips within 2 minutes**. A constant guess manages 5.4 minutes and 19%.

What the Stage 2 reformulation changed. The Stage 1 features, time and weather, capture about 1.5% of the achievable improvement. Route length captures 99.8%. Excluding the route had excluded most of the answer. The trip's own recorded distance remains excluded: route length is learned only from earlier trips on the same route.

The three-factor assumption, tested:

- **Length** — confirmed, and dominant. Duration grows as $\text{length}^{0.85}$: longer trips are ridden faster.
- **Time** — confirmed, but small. On the same route, speed varies by about 14 percentage points across the day, and weekends are 4–5% slower. The pattern reflects trip *purpose* — the morning commute is the fastest time of day — not congestion. It adds about 0.3 percentage points of trips within 2 minutes.
- **Weather** — not supported at daily resolution. Rain makes trips slightly *shorter and faster*, not slower, which corrects the Stage 1 report. Adding weather does not measurably improve predictions.

The loss function. Logging the label and using Huber loss are complementary. Logs remove most of the instability that forgotten rentals cause. Huber removes the bias that remains: even on a log scale, squared error predicts about half a minute too long on ordinary trips, and it loses 6.4 (2024) and 8.2 (2025) percentage points of trips within 2 minutes against Huber. Its outlier sensitivity is about 12 times Huber's.

The hypothesis space. Boosted trees add nothing of practical value. Their differences from the linear model are about 3 seconds of MAE or 0.3 points within 2 minutes, and the sign flips between metrics. The near-linear hypothesis from Stage 1 holds, and the linear model is kept: it was selected by the pre-declared rule, and it is simpler and directly interpretable.

Learning vs arithmetic. Dividing route length by one typical speed (11.5 km/h) already achieves 61 of the 65 percentage points of improvement in typical error. Learning adds a real

but modest refinement: about 3.5 percentage points more trips within 2 minutes. Most of what makes a trip's duration predictable is simply how far it goes.

Limitations:

- **Daily weather is coarse.** A rainy day may have been dry at 17:00. Hourly observations are the last chance for weather to show a speed effect.
- **Route lengths come from history,** so 2.6% of 2025 trips — new stations and rare routes — fall back to cruder estimates.
- **Rider behaviour cannot be predicted.** Stops, detours and forgotten rentals keep the *average* error near 11.5 minutes, even though the *typical* error is under 2.
- **Every model is trained on a 400,000-trip sample.** We did not test whether more data helps.
- **The bootstrap intervals capture uncertainty within one season,** not change between seasons; the 2025 test is our only check on that.
- **Round trips (3.2%) follow a different process,** handled here only by a flag.